

Projet Genie Logiciel - Java

Simulation de Propagation de Feu

Automate cellulaire en plan 2D

Milestones & Issues

Version	1.0
Date	15/05/2026
Statut	Brouillon
Equipe	2 personnes
Milestones	M0 a M5 (6 jalons)

Introduction

Ce document liste l'ensemble des milestones et issues GitHub du projet de simulation de propagation de feu. Il constitue le plan de travail operationnel : chaque milestone correspond a un jalon livrable et auto-suffisant, et chaque issue decrit une tache precise a accomplir pour l'atteindre.

L'objectif est de pouvoir suivre les milestones dans l'ordre, de M0 a M5, sans avoir a revenir en arriere ni anticiper un jalon futur.

Recapitulatif des milestones :

- M0 - Setup & Initialisation (4 issues)
- M1 - Socle du modele (7 issues)
- M2 - Moteur de simulation (5 issues)
- M3 - Parametres avances (4 issues)
- M4 - Interface graphique (8 issues)
- M5 - Finalisation (4 issues)

M0 — Setup & Initialisation (4 issues)

Objectif : mettre en place toutes les fondations du projet avant d'ecrire la premiere ligne de code metier.

Ce milestone doit etre complete en premier. A sa cloture, le depot est operationnel, la structure du projet Java est en place, le document de specification est valide et le diagramme de cas d'utilisation est produit. Aucune fonctionnalite metier n'est implementee a ce stade.

#1 Creer et configurer le depot GitHub

Initialiser le depot GitHub du projet avec toutes les configurations necessaires au travail collaboratif a deux.

- Creer le depot (public ou prive selon les consignes)
- Ajouter un fichier .gitignore adapte a Java et aux outils de build (Maven/Gradle, IDE)
- Rediger un README.md initial : titre du projet, description courte, instructions d'installation et de lancement (a completer au fil du projet)
- Configurer la protection de la branche main (interdire les push directs, imposer les pull requests)
- Creer la branche develop a partir de main
- Definir la convention de nommage des branches : feature/<nom>, fix/<nom>
- Inviter le second membre de l'equipe comme collaborateur

#2 Initialiser la structure du projet Java et le build tool

Mettre en place la structure de repertoires du projet Java et configurer Maven ou Gradle pour que le projet compile et soit executable des le depart.

- Choisir Maven ou Gradle et initialiser le projet (pom.xml ou build.gradle)
- Creer la structure de repertoires standard : src/main/java, src/test/java, src/main/resources
- Definir les packages Java de base : model, view, controller, util
- Ajouter les dependances initiales : JUnit 5 pour les tests, JavaFX si gere via Maven/Gradle
- Verifier que le projet compile sans erreur avec une classe Main vide
- Commiter et pusher la structure sur la branche develop

#3 Rediger le document de specification des fonctionnalites

Produire le document de reference du projet decrivant le modele de simulation, les fonctionnalites attendues, l'architecture cible et les jalons. Ce document sert de contrat entre les membres de l'equipe et de reference tout au long du projet.

- Decrire le modele de simulation : etats des cellules, types de terrain, regles de propagation
- Decrire les parametres environnementaux : vent (8 directions, intensite 0-5), humidite (0-100%)
- Lister toutes les fonctionnalites (F1 a F5) avec leur identifiant
- Decrire l'architecture MVC cible et les principaux composants Java
- Lister les milestones et leur contenu
- Valider le document avec les deux membres de l'equipe avant de le merger sur develop

#4 Produire le diagramme de cas d'utilisation

Realiser le diagramme UML de cas d'utilisation representant toutes les interactions entre les acteurs (Utilisateur, Systeme) et l'application, conformément a la section 5.1 du document de specification.

- Identifier les deux acteurs : Utilisateur et Systeme
- Modeliser les cas d'utilisation Utilisateur : configurer l'environnement, initialiser et lancer, controler la simulation, visualiser via GUI, lancer en mode console, exporter les resultats
- Modeliser les cas d'utilisation Systeme : calculer la propagation a chaque tick, mettre a jour les etats, notifier la vue (pattern Observer)
- Exporter le diagramme en PNG ou PDF et le versionner dans le depot (dossier docs/uml/)

M1 — Socle du modele (7 issues)

Objectif : implémenter toutes les classes du modele metier (couche Model du MVC) sans aucune logique de simulation.

A la cloture de ce milestone, le modele de donnees est complet et testé : on peut creer une grille, lui affecter des cellules avec des etats et des types de terrain, et naviguer dans le voisinage. Le diagramme de classes et le diagramme d'etats-transitions sont produits et versionés.

#5 Implementer l'enumeration CellState

Creer l'enumeration Java CellState representant les six etats possibles d'une cellule, conformement a la section 2.1.1 du document de specification.

- Definir les six constantes : VIDE, SAIN, EN_FEU, BRULE, EAU, ROCHER
- Associer a chaque etat sa couleur d'affichage (code hexadecimal ou constante JavaFX Color) pour usage futur dans la GUI
- Ajouter une methode utilitaire isFlammable() retournant true uniquement pour SAIN
- Ajouter une methode isSpreading() retournant true uniquement pour EN_FEU
- Ecrire les tests unitaires JUnit 5 couvrant isFlammable() et isSpreading()

#6 Implementer l'enumeration CellType

Creer l'enumeration Java CellType representant les cinq types de terrain, chacun portant ses proprietes d'inflammabilite et de duree de combustion, conformement a la section 2.1.2.

- Definir les cinq constantes : FORET, PRAIRIE, BROUSSAILLES, ZONE_HUMIDE, ZONE_URBAINE
- Associer a chaque type un coefficient d'inflammabilite (double entre 0.0 et 1.0)
- Associer a chaque type une duree de combustion en nombre de ticks (int)
- Ajouter des getters : getInflammability(), getBurnDuration()
- Ecrire les tests unitaires verifiant les valeurs de chaque type

#7 Implementer la classe Cell

Creer la classe Java Cell representant une cellule individuelle de la grille, avec son etat courant, son type de terrain et sa duree de combustion restante.

- Attributs : CellState state, CellType type, int remainingBurnTime
- Constructeur : Cell(CellType type) initialisant l'etat a SAIN et remainingBurnTime a type.getBurnDuration()
- Methodes : getState(), getType(), getRemainingBurnTime(), setState(CellState), decrementBurnTime()
- Methode ignite() : passe l'etat a EN_FEU si la cellule est SAIN, sinon ne fait rien
- Methode burnOut() : passe l'etat a BRULE et remet remainingBurnTime a 0
- Ecrire les tests unitaires couvrant les transitions d'etat et la decrementation

#8 Implementer la classe Grid

Créer la classe Java Grid représentant la grille 2D de cellules, avec ses méthodes d'accès, d'initialisation et de calcul du voisinage de Moore.

- Attributs : int width, int height, Cell[][] cells
- Constructeur : Grid(int width, int height) allouant le tableau de cellules
- Méthodes d'accès : getCell(int x, int y), setCell(int x, int y, Cell), isInBounds(int x, int y)
- Méthode getNeighbors(int x, int y) retournant la liste des 8 voisins de Moore (en excluant les cellules hors limites)
- Méthode initRandom(double densityForest, double densityPrairie, ...) peuplant la grille aléatoirement
- Méthode reset() rétablissant la grille à son état initial (memorise à l'initialisation)
- Méthode setFire(int x, int y) plaçant un foyer initial en passant la cellule à EN_FEU
- Écrire les tests unitaires sur getNeighbors (coins, bords, centre), isInBounds et reset

#9 Implémenter la classe Environment

Créer la classe Java Environment agréant les paramètres environnementaux de la simulation. À ce stade, la classe est un simple conteneur de données sans logique de calcul.

- Attributs : WindDirection direction (enum 8 valeurs : N, NE, E, SE, S, SO, O, NO), int windStrength (0 à 5), int humidity (0 à 100)
- Créer l'énumération WindDirection avec ses 8 constantes et un vecteur directionnel associé (dx, dy)
- Constructeur par défaut : pas de vent (N, force 0), humidité 50%
- Constructeur paramétrique et getters/setters avec validation des plages (windStrength clamped entre 0 et 5, humidity entre 0 et 100)
- Écrire les tests unitaires vérifiant la validation des plages

#10 Produire le diagramme de classes

Réaliser le diagramme UML de classes couvrant l'ensemble du modèle métier implémenté dans ce milestone, conformément à la section 5.2 du document de spécification.

- Représenter les classes Cell, Grid, Environment et les énumérations CellState, CellType, WindDirection
- Faire figurer tous les attributs, méthodes et types de retour
- Représenter les relations : Grid contient Cell (composition), Cell utilise CellState et CellType (dépendance), Environment utilise WindDirection
- Laisser des emplacements prévus pour FireSimulator, SimulationController et les vues (à compléter aux milestones suivants)
- Exporter en PNG ou PDF et versionner dans docs/uml/

#11 Produire le diagramme d'états-transitions

Réaliser le diagramme UML d'états-transitions modélisant le cycle de vie complet d'une cellule, conformément à la section 5.2 du document de spécification.

- Représenter les six états : VIDE, SAIN, EN_FEU, BRULE, EAU, ROCHER
- Modéliser les transitions : SAIN -> EN_FEU (inflammation), EN_FEU -> BRULE (combustion terminée)

- Indiquer les gardes sur les transitions : probabilité d'inflammation P, remainingBurnTime = 0
- Indiquer les états terminaux (VIDE, BRULE, EAU, ROCHER) qui ne peuvent plus évoluer
- Exporter en PNG ou PDF et versionner dans docs/uml/

M2 — Moteur de simulation (5 issues)

Objectif : implémenter le moteur de simulation et la vue console, rendant la simulation fonctionnelle en mode headless.

A la cloture de ce milestone, la simulation tourne en ligne de commande : on peut la lancer, la mettre en pause, avancer tick par tick et l'arrêter. Les règles de propagation de base sont appliquées (sans vent ni humidité). Le diagramme de séquence est produit.

#12 Implémenter la classe FireSimulator

Créer la classe Java FireSimulator constituant le moteur de simulation. Elle applique les règles de propagation à chaque tick sur la grille, sans prise en compte du vent ni de l'humidité (paramètres avancés réservés à M3).

- Attributs : Grid grid, Environment environment
- Méthode tick() : parcourir toutes les cellules EN_FEU, calculer la propagation vers chaque voisin SAIN, decrementer le remainingBurnTime des cellules EN_FEU, passer les cellules dont remainingBurnTime = 0 à l'état BRULE
- Méthode computeInflammationProbability(Cell source, Cell target) : retourner à ce stade uniquement le coefficient d'inflammabilité du type de terrain cible (sans vent ni humidité)
- La mise à jour des états doit être faite sur une copie de la grille (double buffering) pour éviter les effets de bord dans le même tick
- Écrire les tests unitaires : propagation basique, extinction après durée de combustion, absence de propagation sur EAU et ROCHER

#13 Implémenter SimulationController

Créer la classe Java SimulationController orchestrant la boucle de simulation et exposant les commandes de contrôle (start, pause, step, stop, reset).

- Attributs : FireSimulator simulator, SimulationState state (enum : IDLE, RUNNING, PAUSED), int currentTick, int tickDelay (ms)
- Méthode start() : lancer la boucle de simulation dans un thread séparé, appeler simulator.tick() à intervalle régulier (tickDelay)
- Méthode pause() : suspendre la boucle sans réinitialiser l'état
- Méthode step() : exécuter exactement un tick manuellement (utilisable uniquement en état PAUSED ou IDLE)
- Méthode stop() : arrêter la boucle et réinitialiser le tick courant à 0
- Méthode reset() : appeler grid.reset() et repasser en état IDLE
- Méthode setTickDelay(int ms) : modifier la vitesse de simulation (F2.3)
- Écrire les tests unitaires sur les transitions d'état du contrôleur

#14 Implémenter ConsoleView

Créer la classe Java ConsoleView assurant le rendu ASCII de la grille dans le terminal et l'affichage des statistiques à chaque tick, conformément aux fonctionnalités F5.2 et F5.3.

- Méthode render(Grid grid) : afficher la grille sous forme de caractères ASCII codés par état (ex : '.' VIDE, 'V' SAIN, 'F' EN_FEU, 'X' BRULE, '~' EAU, 'R' ROCHER)

- Methode printStats(int tick, Grid grid) : afficher sur une ligne le tick courant, le nombre de cellules saines, en feu et brulees, et le taux de destruction en pourcentage
- Afficher une legende des caracteres au demarrage de la simulation
- Effacer le terminal entre chaque tick pour un rendu fluide (ANSI escape codes ou equivalent)

#15 Implementer le lancement en mode headless via CLI

Permettre le lancement de la simulation en ligne de commande sans interface graphique, conformément a la fonctionnalite F5.1.

- Analyser les arguments CLI dans la classe Main : --width, --height, --ticks (nombre de ticks max), --delay (ms par tick)
- Instancier Grid, Environment, FireSimulator, SimulationController et ConsoleView a partir des arguments
- Placer un foyer initial par default au centre de la grille si aucun foyer n'est specifie
- Arrêter automatiquement la simulation apres le nombre de ticks max ou si plus aucune cellule n'est EN_FEU
- Afficher un message de fin avec les statistiques globales (taux de destruction final, nombre de ticks ecoules)

#16 Produire le diagramme de sequence

Realiser le diagramme UML de sequence modelisant le deroulement complet d'un tick de simulation, conformément a la section 5.2 du document de specification.

- Représenter les participants : SimulationController, FireSimulator, Grid, Cell, ConsoleView
- Modeliser la sequence complete : SimulationController appelle tick() sur FireSimulator, qui itere sur les cellules EN_FEU, appelle computeInflammationProbability(), met a jour les etats via Cell, puis notifie ConsoleView
- Faire figurer le double buffering (lecture sur l'ancienne grille, ecriture sur la nouvelle)
- Exporter en PNG ou PDF et versionner dans docs/uml/

M3 — Parametres avances (4 issues)

Objectif : integrer le vent, l'humidite et les types de terrain dans le calcul de propagation, et permettre leur configuration via la console.

A la cloture de ce milestone, la simulation est realiste : le feu se propage preferentiellement dans la direction du vent, l'humidite freine l'inflammation, et chaque type de terrain a un comportement propre. Les parametres sont configurables avant et pendant la simulation via la console.

#17 Integrer le vent dans le calcul de la probabilite d'inflammation

Modifier la methode `computeInflammationProbability()` de `FireSimulator` pour tenir compte de la direction et de l'intensite du vent dans le calcul de la probabilite qu'une cellule voisine s'enflamme.

- Calculer l'angle entre la direction du vent et la direction source -> cible
- Appliquer un facteur multiplicateur : augmentation si la cible est dans la direction du vent, diminution si elle est a contre-vent (utiliser une fonction cosinus ou un tableau de facteurs par direction)
- L'intensite du vent (0 a 5) module l'amplitude de cet effet : a intensite 0 le vent n'a aucun effet
- Ecrire les tests unitaires verifiant que la probabilite est plus elevee dans la direction du vent qu'a contre-vent, a intensite egale

#18 Integrer l'humidite dans le calcul de la probabilite d'inflammation

Modifier la methode `computeInflammationProbability()` de `FireSimulator` pour appliquer un facteur reducteur lie au taux d'humidite de l'environnement.

- Appliquer un facteur reducteur lineaire : $P = P_base * (1 - humidity / 100.0)$
- A 100% d'humidite, aucune cellule ne peut s'enflammer ($P = 0$) quel que soit le type de terrain
- A 0% d'humidite, le facteur est neutre ($P = P_base$)
- Ecrire les tests unitaires verifiant les cas extremes (0%, 100%) et un cas intermediaire

#19 Verifier l'integration des types de terrain dans la propagation

S'assurer que les coefficients d'inflammabilite et les durees de combustion definis dans `CellType` sont correctement utilises par `FireSimulator` dans tous les scenarios de propagation.

- Verifier que `computeInflammationProbability()` utilise bien `cellType.getInflammability()` comme base du calcul
- Verifier que le `remainingBurnTime` est initialise a `cellType.getBurnDuration()` a l'inflammation
- Ecrire des tests comparatifs : a meme vent et humidite, une PRAIRIE doit s'enflammer plus facilement qu'une ZONE_HUMIDE
- Ecrire un test verifiant qu'une cellule FORET brule plus longtemps qu'une cellule PRAIRIE

#20 Permettre la configuration des parametres via la console

Permettre à l'utilisateur de configurer le vent, l'humidité et le pas de temps avant et pendant la simulation via des commandes en ligne, conformément aux fonctionnalités F2.1, F2.2 et F2.3.

- Implémenter un mini-interpréteur de commandes dans ConsoleView : lire les entrées utilisateur en parallèle de la boucle de simulation (thread séparé ou mode interactif entre les ticks)
- Commande 'wind <direction> <intensité>' : ex. 'wind N 3' pour vent Nord force 3
- Commande 'humidity <valeur>' : ex. 'humidity 60' pour 60% d'humidité
- Commande 'speed <ms>' : ex. 'speed 500' pour 500ms par tick
- Commande 'help' : afficher la liste des commandes disponibles
- Valider les entrées et afficher un message d'erreur explicite en cas de valeur invalide

M4 — Interface graphique (8 issues)

Objectif : implementer la GUI complete (JavaFX ou Swing) permettant de visualiser et controler la simulation de maniere interactive.

A la cloture de ce milestone, l'application est pleinement utilisable via son interface graphique. Toutes les fonctionnalites F3 et F4 sont implementees. La GUI et la console restent toutes deux fonctionnelles (deux modes de lancement distincts).

#21 Mettre en place la fenetre principale et l'architecture MVC graphique

Creer la fenetre principale de l'application JavaFX/Swing et connecter les composants graphiques au modele via le pattern MVC, en implementant le pattern Observer pour les mises a jour de la vue.

- Creer la classe MainWindow (JFrame ou Stage JavaFX) structurant la mise en page : zone de grille a gauche, panneau de controle et statistiques a droite
- Implementer le pattern Observer : FireSimulator (ou SimulationController) notifie les vues apres chaque tick via une interface SimulationListener
- GridView, StatisticsPanel et ChartView s'enregistrent comme listeners et se mettent a jour a chaque notification
- Gerer le thread JavaFX/Swing (Platform.runLater ou SwingUtilities.invokeLater) pour les mises a jour graphiques depuis le thread de simulation

#22 Implementer GridView

Creer le composant graphique GridView assurant le rendu visuel de la grille en temps reel avec un code couleur par etat de cellule, conformement a la fonctionnalite F4.1.

- Dessiner chaque cellule sous forme de rectangle colore selon son CellState (couleurs definies dans CellState)
- La taille d'une cellule s'adapte dynamiquement a la taille de la fenetre et aux dimensions de la grille
- Redessiner uniquement les cellules dont l'etat a change depuis le dernier tick (optimisation)
- Permettre le clic sur une cellule pour afficher ses informations (etat, type, remainingBurnTime) dans une info-bulle ou une zone de texte

#23 Implementer le zoom et le deplacement dans la grille

Permettre a l'utilisateur de zoomer et de se deplacer dans la grille pour les simulations sur de grandes grilles, conformement a la fonctionnalite F4.4.

- Zoom in / out via la molette de la souris (ou boutons + / -), avec un facteur de zoom entre 0.5x et 5x
- Deplacement (pan) via cliquer-glisser sur la grille
- Bouton 'Reset zoom' pour revenir a la vue globale ajustee a la fenetre
- Afficher les coordonnees (x, y) de la cellule survolee par la souris

#24 Implementer le panneau de statistiques en temps reel

Créer le composant `StatisticsPanel` affichant les statistiques de la simulation mises à jour à chaque tick, conformément à la fonctionnalité F4.2.

- Afficher : tick courant, nombre de cellules SAIN / EN_FEU / BRULE, taux de destruction (BRULE / total cellules végétalisées en %)
- Afficher l'état courant de la simulation (IDLE, RUNNING, PAUSED)
- Afficher les paramètres environnementaux actifs : direction et intensité du vent, taux d'humidité, vitesse (ms/tick)
- Mettre à jour l'affichage à chaque notification du `SimulationListener`

#25 Implémenter la légende et l'indicateur de vent

Créer les composants affichant la légende des états/types de terrain et un indicateur visuel de la direction et de l'intensité du vent, conformément aux fonctionnalités F4.3 et F4.5.

- Légende : afficher un carré coloré suivi du libellé pour chacun des six états de cellule
- Indicateur de vent : dessiner une flèche pointant dans la direction du vent, dont la longueur ou l'épaisseur varie selon l'intensité (0 = flèche absente ou grisee, 5 = flèche épaisse)
- L'indicateur se met à jour immédiatement quand le vent est modifié pendant la simulation

#26 Implémenter `ChartView` - courbe d'évolution

Créer le composant `ChartView` affichant en temps réel la courbe du nombre de cellules EN_FEU au fil des ticks, conformément à la fonctionnalité F4.6.

- Afficher une courbe linéaire : axe X = ticks écoulés, axe Y = nombre de cellules EN_FEU
- La courbe se met à jour à chaque tick en ajoutant le nouveau point
- Adapter l'échelle de l'axe Y automatiquement au maximum observé
- Afficher également les courbes SAIN et BRULE en couleurs distinctes pour visualiser l'évolution globale

#27 Implémenter les contrôles de simulation dans la GUI

Ajouter dans la GUI les boutons de contrôle de la simulation et les champs de paramétrage, conformément aux fonctionnalités F3.1 à F3.5.

- Boutons : Démarrer, Pause/Reprendre, Avancer d'un tick (Step), Arrêter, Réinitialiser
- Griser les boutons incohérents avec l'état courant (ex : Step grise si RUNNING, Démarrer grise si RUNNING)
- Champs de paramétrage : direction du vent (menu déroulant 8 directions), intensité du vent (slider 0-5), humidité (slider 0-100), vitesse en ms/tick (champ numérique ou slider)
- Les modifications des paramètres prennent effet immédiatement, même si la simulation est en cours (F3.5)
- Champ pour définir les coordonnées du foyer initial avant le lancement

#28 Permettre le placement interactif du terrain et des foyers via la GUI

Permettre à l'utilisateur de dessiner les types de terrain et de placer les foyers d'incendie directement sur la grille graphique avant le lancement de la simulation, conformément aux fonctionnalités F1.3 et F1.4.

- Mode edition (actif uniquement quand la simulation est IDLE) : clic gauche sur une cellule pour lui assigner le type de terrain selectionne dans un menu
- Menu de selection du type de terrain a peindre : FORET, PRAIRIE, BROUSSAILLES, ZONE_HUMIDE, ZONE_URBAINE, EAU, ROCHER
- Clic droit sur une cellule pour y placer un foyer d'incendie (passe la cellule a EN_FEU)
- Peinture par glisser-deposer (cliquer et faire glisser pour peindre plusieurs cellules d'un coup)

M5 — Finalisation (4 issues)

Objectif : consolider la qualité du projet par les tests unitaires, la mise à jour des diagrammes UML et la rédaction de la documentation technique finale.

A la cloture de ce milestone, le projet est complet et livrable. Les tests couvrent les composants critiques, les diagrammes UML reflètent fidèlement l'implémentation réelle, et le README permet à n'importe qui de lancer le projet sans aide extérieure.

#29 Ecrire les tests unitaires sur les règles de propagation

Rédiger une suite de tests JUnit 5 couvrant exhaustivement la logique de calcul de la probabilité d'inflammation et les transitions d'états dans FireSimulator.

- Tester `computeInflammationProbability()` : cas sans vent, avec vent dans la direction cible, avec vent à contre-vent, avec humidité 0%, 50%, 100%
- Tester que la propagation ne se produit jamais vers une cellule EAU, ROCHER, BRULE ou VIDE
- Tester le passage `EN_FEU` -> `BRULE` après exactement `burnDuration` ticks pour chaque `CellType`
- Tester le double buffering : une cellule qui s'enflamme au tick N ne propage pas au tick N
- Tester qu'une simulation sans cellule `EN_FEU` ne modifie pas la grille

#30 Ecrire les tests unitaires sur Grid et SimulationController

Rédiger les tests JUnit 5 couvrant les méthodes critiques de Grid et les transitions d'état de SimulationController.

- Grid - `getNeighbors()` : tester les 8 cas (coin haut-gauche, coin haut-droit, coin bas-gauche, coin bas-droit, bord haut, bord bas, bord gauche, bord droit, cellule centrale)
- Grid - `reset()` : vérifier que la grille retrouve exactement son état initial après plusieurs ticks
- Grid - `isInBounds()` : tester les limites exactes et les valeurs hors limites
- SimulationController : tester les transitions `IDLE` -> `RUNNING` -> `PAUSED` -> `RUNNING` -> `IDLE` et `RUNNING` -> `IDLE` via `stop()`
- SimulationController - `step()` : vérifier qu'un seul tick est exécuté et que l'état reste `PAUSED`/`IDLE`

#31 Mettre à jour le diagramme de classes final

Produire la version finale du diagramme de classes reflétant fidèlement l'implémentation réelle de l'ensemble du projet, en complétant et corrigeant la version produite en M1.

- Ajouter les classes implémentées en M2, M3 et M4 : `FireSimulator`, `SimulationController`, `ConsoleView`, `MainWindow`, `GridView`, `StatisticsPanel`, `ChartView`
- Ajouter l'interface `SimulationListener` et ses implémentations
- Vérifier la cohérence de toutes les relations (composition, aggrégation, dépendance, implémentation)
- Vérifier que tous les attributs et méthodes publics sont représentés
- Exporter en PNG ou PDF et versionner dans `docs/uml/` en remplaçant la version précédente

#32 Rediger la documentation technique finale (README)

Compléter et finaliser le README.md du depot GitHub pour qu'il constitue une documentation technique complete permettant a n'importe quel developpeur de comprendre, installer et lancer le projet.

- Description du projet : contexte, objectif, fonctionnalites principales
- Prerequis : Java 17+, Maven ou Gradle, JavaFX si necessaire
- Instructions d'installation : cloner le depot, installer les dependances
- Instructions de lancement : mode GUI (commande Maven/Gradle), mode headless (arguments CLI disponibles et leur description)
- Description de l'architecture : schema simplifie MVC, role de chaque package
- Instructions pour lancer les tests unitaires
- Lien vers les diagrammes UML dans docs/uml/